D09 Reference

D09 Instruction Set

Core Instruction Formats

31 26	25 21	20 16	15 13	12 7	6 2	1 0	
000000	rs2	rs1	funct3	opcode	rd	00	R-type
imm[10:0]		rs1	funct3	opcode	rd	00	I-type
imm[11:6]	rs2	rs1	funct3	opcode	imm[5:1]	00	B-type

D09 Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	Description (C)	Note
`add`	ADD	R	`001100`	`0x0`	`rd = rs1 + rs2`	
`sub`	SUB	R	`001100`	`0x3`	`rd = rs1 - rs2`	
`xor`	XOR	R	`001100`	`0x4`	`rd = rs1 ^ rs2`	
`or`	OR	R	`001100`	`0x6`	`rd = rs1 | rs2`	
`and`	AND	R	`001100`	`0x7`	`rd = rs1 & rs2`	
`sll`	Shift Left Logical	R	`001100`	`0x1`	`rd = rs1 << rs2[4:0]`	
`srl`	Shift Right Logical	R	`001100`	`0x5`	`rd = rs1 >> rs2[4:0]`	zero-extends
`slt`	Set Less Than	R	`001100`	`0x2`	`rd = (rs1 < rs2)?1:0`	
`addi`	ADD Immediate	I	`000100`	`0x0`	`rd = rs1 + imm`	
`xori`	XOR Immediate	I	`000100`	`0x4`	`rd = rs1 ^ imm`	
`ori`	OR Immediate	I	`000100`	`0x6`	`rd = rs1 | imm`	
`andi`	AND Immediate	I	`000100`	`0x7`	`rd = rs1 & imm`	
`slli`	Shift Left Logical Imm	I	`000100`	`0x1`	`rd = rs1 << imm[4:0]`	imm[10:5]=0
`srli`	Shift Right Logical Imm	I	`000100`	`0x5`	`rd = rs1 >> imm[4:0]`	imm[10:5]=0
`slti`	Set Less Than Imm	I	`000100`	`0x2`	`rd = (rs1 < imm)?1:0`	
`lb`	Load Byte	I	`000001`	`0x0`	`rd = M[rs1+imm][0:7]`	msb-extends
`lw`	Load Word	I	`000001`	`0x2`	`rd = M[rs1+imm][0:31]`	
`beq`	Branch ==	B	`011000`	`0x0`	`if(rs1 == rs2) PC += imm`	
`bne`	Branch !=	B	`011000`	`0x1`	`if(rs1 != rs2) PC += imm`	
`blt`	Branch <	B	`011000`	`0x4`	`if(rs1 < rs2) PC += imm`	
`bge`	Branch $\geq$	B	`011000`	`0x5`	`if(rs1 >= rs2) PC += imm`	

Registers

Register	ABI Name	Description	Saver
`x0`	`zero`	Zero constant	—
`x1`	`ra`	Return address	Caller
`x2`	`sp`	Stack pointer	Callee
`x3`	`gp`	Global pointer	—
`x4`	`tp`	Thread pointer	—
`x5-x7`	`t0-t2`	Temporaries	Caller
`x8`	`s0 / fp`	Saved / frame pointer	Callee
`x9`	`s1`	Saved register	Callee
`x10-x11`	`a0-a1`	Fn args/return values	Caller
`x12-x17`	`a2-a7`	Fn args	Caller
`x18-x27`	`s2-s11`	Saved registers	Callee
`x28-x31`	`t3-t6`	Temporaries	Caller